Synchronous Design

Lecture 04

Josh Brake
Harvey Mudd College

Outline

- Review of basic synchronous design
- Review of the dynamic discipline
- Synchronizers and metastability
- FSM design steps
- **Activity:** design a level-to-pulse converter
- Diode and transistor review

Learning Objectives

By the end of this lecture you should be able to…

- Recall the dynamic discipline and timing specs for designing synchronous digital systems.
- Properly condition asynchronous signals using synchronizers.
- Design an FSM from a written spec: state transition diagram, table, and Verilog.
- Judge when a problem needs an FSM and when a few flops will do.
- Write a testbench that does not sample its own inputs on a clock edge.
- Recall how to use transistors to drive large currents.

Synchronous Digital Systems

- Timing problems are usually the #1 source of difficult bugs
- We can almost completely eliminate the timing problems with a synchronous discipline
  - Like digital vs. analog: digital is a subset of analog
  - Synchronous is a subset of asynchronous timing methodologies
  - Limiting choice makes design easier to understand and avoid sneaky bugs
- Also, will make testability easier (we'll see that later)

Basic Synchronous Design Rules

- Use only one clock signal (named something clear like `clk`)
- Use only flip-flops as state elements (no latches!)
- Put this clock signal into the clock terminal of every flip-flop in the system.

Some common gotchas

Q: How do we begin in a known state?

A: Use a reset signal.

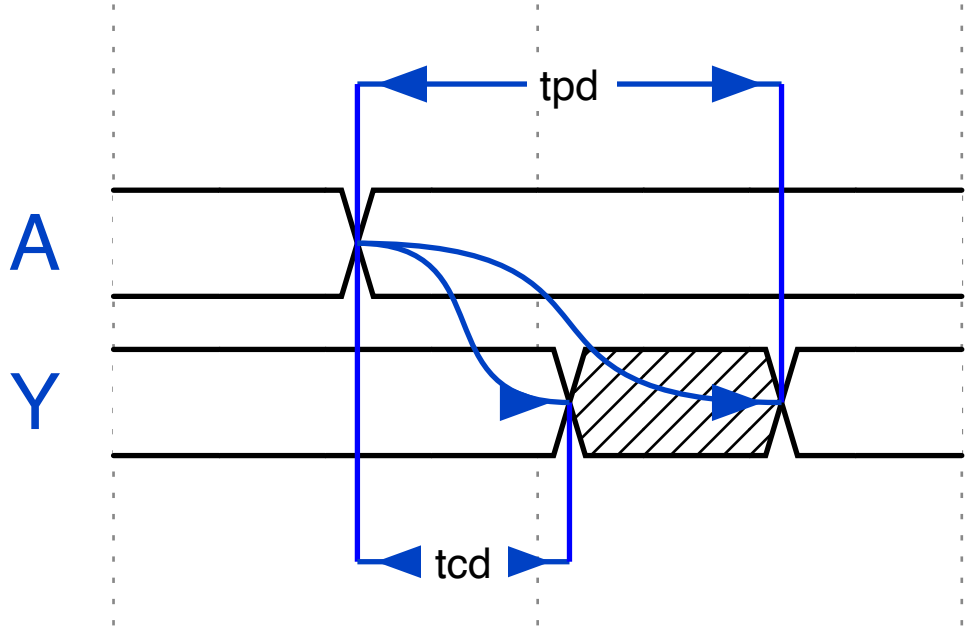
Q: How do we avoid changing the contents of a flip-flop on every clock cycle?

A: Use an enable.

Dynamic Discipline Review

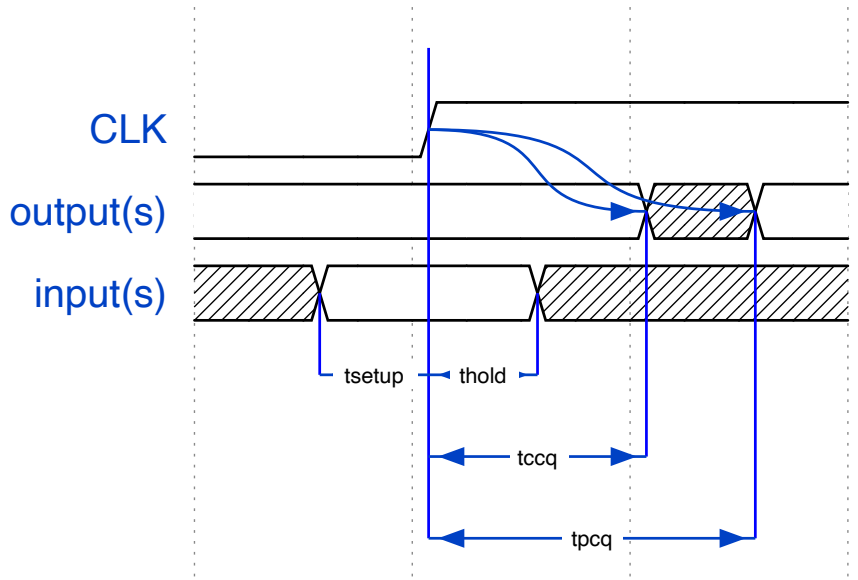
Propagation delay (t_{pd}) – the maximum time from when an input changes until the output(s) reach their final value.

Contamination delay (t_{cd}) – the minimum time from when an input changes until any output starts to change its value.



Dynamic Discipline Review: Sequential Logic

- **Propagation Clock-to-Q** (t_{pcq}) – upper bound on the time from the rising edge of the clock until the output changes.
- **Contamination Clock-to-Q** (t_{ccq}) – lower bound on the time from the rising edge of the clock until the output changes.
- **Setup time** (t_{setup}) – the amount of time an input to a flop must be stable before the clock edge.
- **Hold time** (t_{hold}) – the amount of time an input to a flop must be stable after the clock edge.



Synchronous Timing Constraints

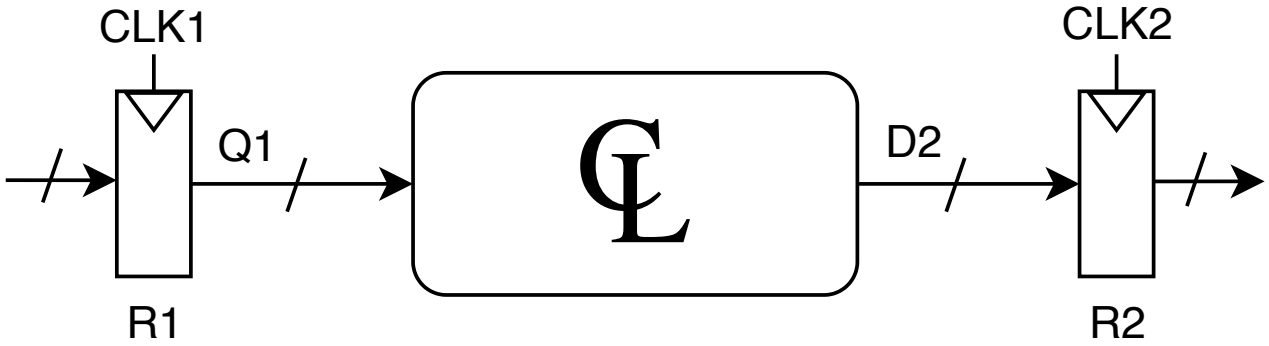
Setup Time Constraint

$$t_{pcq} + t_{pd} + t_{skew} + t_{setup} \leq T_c$$

Hold Time Constraint

$$t_{ccq} + t_{cd} \geq t_{hold} + t_{skew}$$

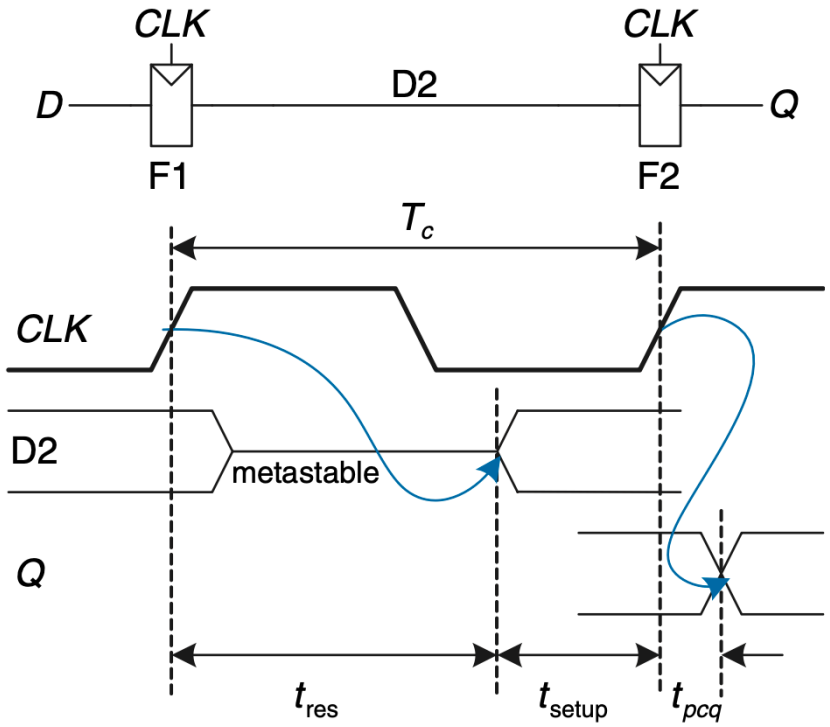
Understanding Timing Constraints



Synchronizers

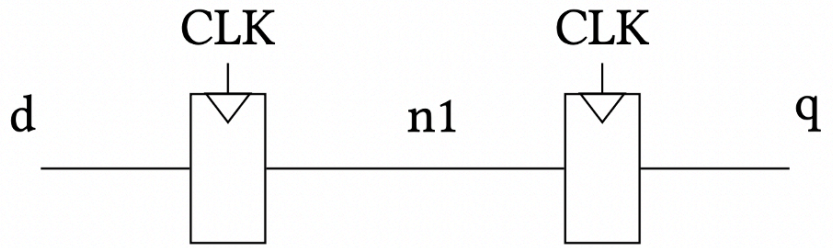
Synchronizer

- The world is asynchronous – how can we cope? Synchronizers!
- Simplest case is a 2-stage synchronizer made of two flops in series.
- If the output of flop F1 goes metastable, we have some time for it to resolve before the next clock edge and the second flop F2.
- This avoids passing metastable inputs out to combinational logic.



Synchronizer

```
 1 module sync(input   logic clk,
 2               input   logic d,
 3               output  logic q);
 4
 5     logic n1;
 6
 7     always_ff @(posedge clk)
 8     begin
 9       n1 <= d;
10       q <= n1;
11     end
12 endmodule
```



Another Synchronizer (?)

What if I replace the non-blocking assignments with blocking assignments?

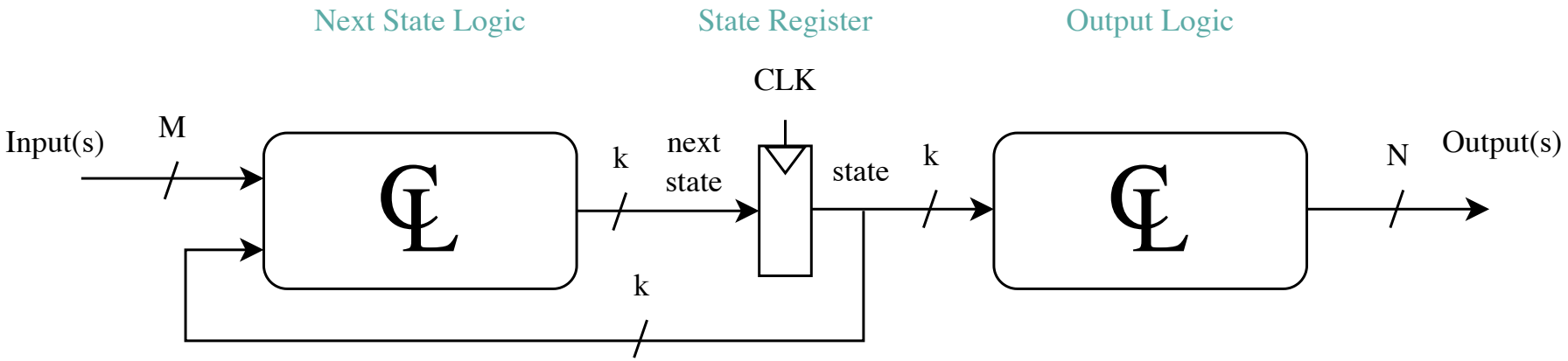
What logic does this imply?

No, this implies only a single flop. The synthesizer will see that n1 gets d and then q gets n1 before the block exits and just eliminate the unnecessary intermediate signal.

```
1  module sync(input   logic clk,
2              input   logic d,
3              output  logic q);
4
5      logic n1;
6
7      always_ff @(posedge clk)
8      begin
9        n1 = d;
10       q = n1;
11     end
12 endmodule
```

Finite State Machine (FSM) Review

FSM Design


Next State Logic
State Register
Output Logic
CLK
Input(s)
M
k
next
state
state
k
N
Output(s)
k

FSM Design Process

1. List System Parameters (inputs & outputs)
2. Sketch State Transition Diagram
3. Write State Transition Table
4. Choose Encoding Scheme
5. Write Boolean Next State Equations
6. Write Output Table
7. Implement Logic

FSM Activity

Strobe Signal Generator (Level-to-pulse converter)

You have been tasked with creating circuitry for a single photon detector. When a photon arrives, it generates a pulse of a random length. We want to convert this random-length pulse to an output pulse of a fixed duration whenever a photon hits the detector.

Your task (should you choose to accept it): Design a system which generates a pulse for a single clock cycle when the input goes from low to high. Add a synchronizer to reduce the probability that the input will cause metastability.

How this is going to work

You are designing this one, not watching me design it.

Step	Time	Format
1. List inputs and outputs	2 min	on your own
2. Sketch the state transition diagram	5 min	in pairs
3. Two pairs put their diagram on the board	4 min	all of us
4. Write the Verilog	5 min	in pairs
5. Add the synchronizer the spec asked for	5 min	all of us
6. Find the bugs in my testbench	3 min	in pairs

Slides stay blank until we have talked about them. Resist the urge to look ahead.

Step 1: List the Specifications

On your own, 2 minutes. What goes in, what comes out?

Inputs

- Photon signal: L

Outputs

- pulse: P

Step 2: Sketch the State Transition Diagram

In pairs, 5 minutes. On your handout, draw the states and the transitions between them.

Some things to settle as you go:

- How many states do you need? Why can't you do it with fewer?
- Is P a function of the state alone, or of the state *and* L?
- What happens if L stays high for a hundred cycles?

Step 3: Put It on the Board

Two pairs, 4 minutes. Draw your diagram. We are looking for the *differences*.

Two Designs. Which One Meets the Spec?

3 states, Moore

```
assign P = (state == S1);
```

P depends on the state alone. Three states is the *minimum*: "idle" and "L is high, already pulsed" give the same output but behave differently, so they cannot be one state.

2 states, Mealy

```
assign P = (state == S0) & L;
```

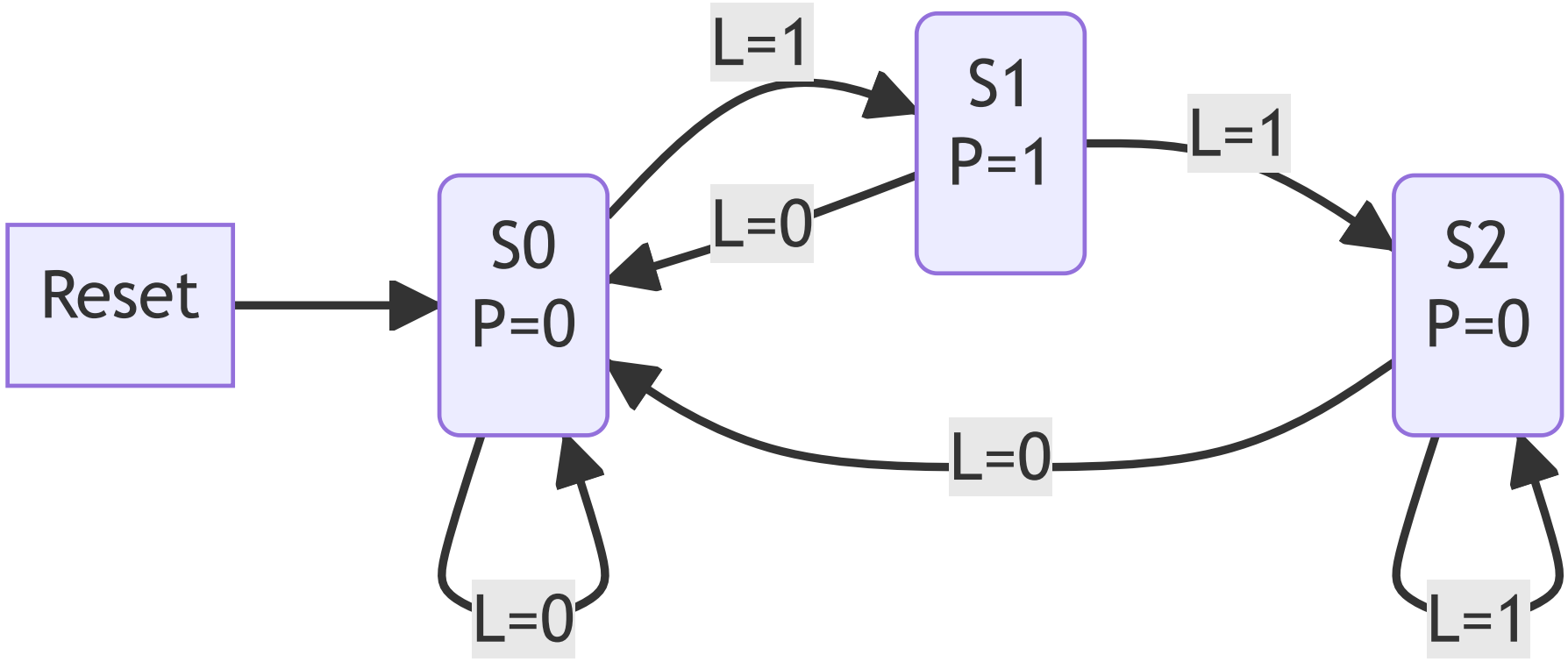
One fewer flop, and P responds a cycle sooner.

The spec asked for a pulse of **fixed duration**. Does the Mealy version give you one?

Not by itself. P follows L through combinational logic, so it rises the instant the photon does and falls at the next clock edge – a width anywhere between 0 and T_c, depending on when the photon happened to arrive.

Now add the synchronizer the spec *also* asked for: L can only move just after a clock edge, so the width comes back to roughly T_c. The two halves of the spec were talking to each other all along. P is still combinational, though, so its edges are not clock-aligned – for a strobe driving an external instrument, ship the Moore version.

State Transition Diagram



State Transition Table

Current State	L	Next State
S0	0	S0
S0	1	S1
S1	0	S0
S1	1	S2
S2	0	S0
S2	1	S2

Output Logic

P=1 when in state S1.

Step 4: Write the Verilog

In pairs, 5 minutes. Five elements:

1. Inputs and outputs
2. Internal signal definition
3. State register: `always_ff` block. Make sure you have a reset!
4. Next state logic: `always_comb` block or `assign` statements.
5. Output logic: `always_comb` block or `assign` statements

FSM Verilog Template

Module and signal declaration.

```
// Produces a one clock cycle pulse on P each time L goes low to high.

module level_to_pulse_converter(
    input  logic clk, reset,
    input  logic L,
    output logic P
);

    logic [2:0] state, nextstate;
    parameter S0 = 3'b001;
    parameter S1 = 3'b010;
    parameter S2 = 3'b100;

    // Or, instead of the four lines above:
    //   typedef enum logic [1:0] {S0, S1, S2} statetype;
    //   statetype state, nextstate;
 ...
```

FSM Verilog Template

State register.

```
...
    // State register
   always_ff @(posedge clk, posedge reset)
       if (reset)  state <= S0;
       else        state <= nextstate;

...
```

Your Turn: Next State and Output Logic

Fill in the case body from your state transition table.

```
    // Next state logic
    always_comb
        case (state)
            S0:
            S1:
            S2:
            default:
        endcase

    // Output logic
    assign P =
```

What does the `default` case buy you here? Do you need it?

Next State and Output Logic

```
 1  ...
 2     // Next state logic
 3     always_comb
 4         case (state)
 5             S0: if(L) nextstate = S1;
 6                 else  nextstate = S0;
 7             S1: if(L) nextstate = S2;
 8                 else  nextstate = S0;
 9             S2: if(L) nextstate = S2;
10                 else  nextstate = S0;
11             default: nextstate = S0;
12         endcase
13
14     // Output logic
15     assign P = (state == S1);
16 endmodule
```

The `default` recovers from the five illegal one-hot states. Without it, `always_comb` would infer a latch.

Step 5: We Forgot Something

Reread the spec:

> Add a synchronizer to reduce the probability that the input will cause metastability.

Where does it go, and what does it cost you?

Two flops between the pad and the FSM's L input. Without it, a photon arriving near a clock edge can drive the state register metastable and land the FSM in an illegal one-hot state.

The cost is **two clock cycles of latency** – exactly the trade you make on the keypad input in Lab 3.

Tempting, But Wrong

The synchronizer's internal node n1 is a one-cycle-delayed copy of d, and q is a two-cycle-delayed copy. So why not skip the FSM entirely?

```
1      // ... from the sync module a few slides back
2      always_ff @(posedge clk)
3        begin
4          n1 <= d;
5          q  <= n1;
6        end
7
8      assign P = n1 & ~q;   // one cycle wide when d rises. Ship it?
```

No. n1 is only *one* flop deep, so it can still be metastable – and you just fed it into combinational logic, which is the exact thing the synchronizer existed to prevent.

The Three-Flop Edge Detector

Synchronize first, *then* edge-detect. One more flop buys back the safety.

```
    logic s1, s2, s3;

    always_ff @(posedge clk, posedge reset)
      if (reset) {s3, s2, s1} <= 3'b000;
      else       {s3, s2, s1} <= {s2, s1, L};

    assign P = s2 & ~s3;   // s2 is synchronized; s3 is s2 delayed one cycle
```

Same behavior as our FSM, no state encoding to get wrong.

Which One Would You Build?

FSM + synchronizer

- 5 flops
- ~30 lines
- Generalizes: add states as the spec grows

Three-flop edge detector

- 3 flops
- 3 lines
- Does exactly one thing

Count the logic cells each one costs. (We will do this properly next lecture.)

Roughly 5 LCs versus 4 – close to a wash on area. The real win is the 3 lines you cannot get wrong. Recognizing when you *don't* need an FSM is the skill.

Writing a Testbench

Steps to create a testbench

1. Create clock signal which toggles continuously for any synchronous elements.
2. Initial statement to apply reset and set inputs to desired initial values.
3. Another initial block to apply input signals.

Don’t apply signals on a clock edge! (e.g., if you are using a clock period of 10 timesteps, don’t apply your inputs at multiples of 10.)

Step 6: Bug Hunt

What is wrong with this testbench?

```
1  `timescale 1ns/1ns
2
3  module l2p_tb();
4    logic clk, reset;
5    logic L, P;
6
7    level_to_pulse_converter l2pc(clk, reset, L, P, debug);
8
9    always begin
10     clk = 1; #5; clk = 0; #5;
11   end
12
13   initial begin
14     reset = 1; #44; reset = 0;
15   end
16
17   initial begin
18     L = 0; #12; L = 1; #12;
19     L = 0; #44; L = 1; #20; L = 0; #32; L = 1;
20   end
21 endmodule
```

1. Five connections to a four-port module and debug is never declared. (2) The last transition lands at t=120, right on a rising edge.

Testbench Code

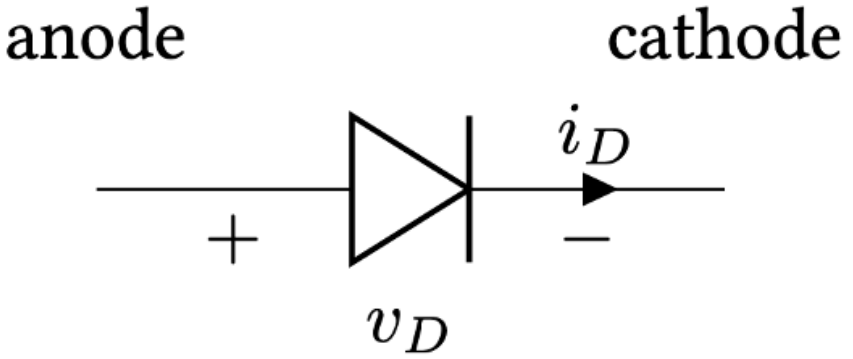
```
1  // Compiler directive to set unit and resolution.
2  `timescale 1ns/1ns
3
4  module l2p_tb();
5    logic clk, reset;
6    logic L, P;
7
8    // Instantiate device under test (DUT)
9    level_to_pulse_converter l2pc(clk, reset, L, P);
10
11   // Clock. Rising edges land on multiples of 10 ns.
12   always begin
13     clk = 1; #5; clk = 0; #5;
14   end
15  ...
```

Testbench Code, cont.

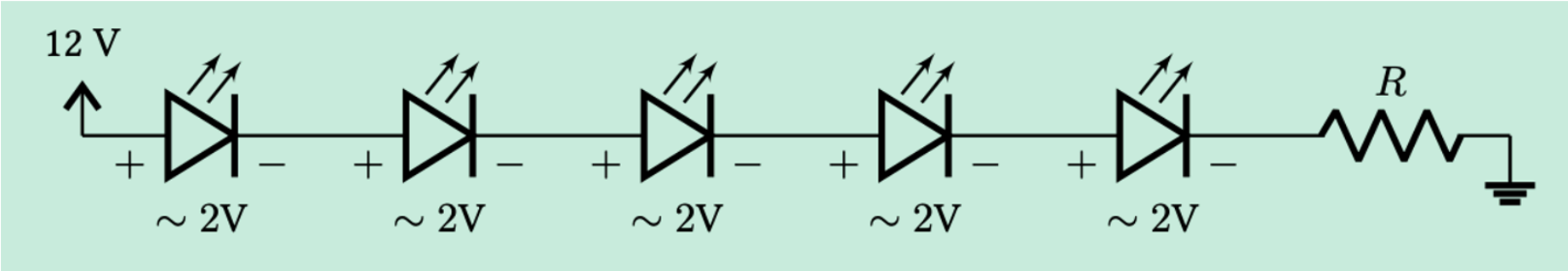
```
  ...
   // Release reset off a clock edge.
   initial begin
     reset = 1; #22; reset = 0;
   end

   // Every transition is offset from the edges at multiples of 10 ns,
   // so no flop ever samples a changing input.
   initial begin
     L = 0;
     #32; L = 1;   // short pulse ...
     #12; L = 0;   // ... gives one cycle of P
     #24; L = 1;   // long pulse ...
     #44; L = 0;   // ... still one cycle of P
     #16; L = 1;
   end
endmodule
```

Diode and Transistor Review

- $i_D = i_0 \left(\exp\left(\frac{v_D}{n \cdot v_T} \right) - 1 \right)$
- n and i_0 are scaling factors and v_T is the thermal voltage which is $v_T = kT/q$ (≈ 26 mV at room temperature).
- For a silicon diode, $v_{on} \approx 0.7V$ and for an LED $v_{on} \approx \ 1.7 - 2.1V$



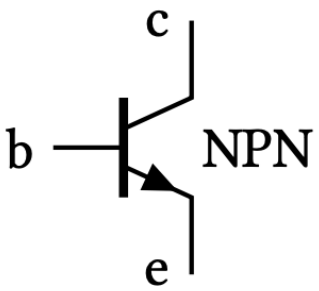
How many LEDs can you light up in series from a 12 V source?



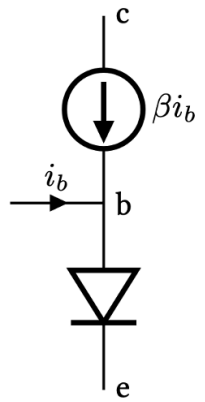
$$i_D = \frac{12 - 5 \cdot 2}{R} = \frac{2}{R}$$

Transistors

Used to pull load low. (i.e., connect load to ground)



NPN Symbol



BJT small signal model

Driving a load with an NPN transistor

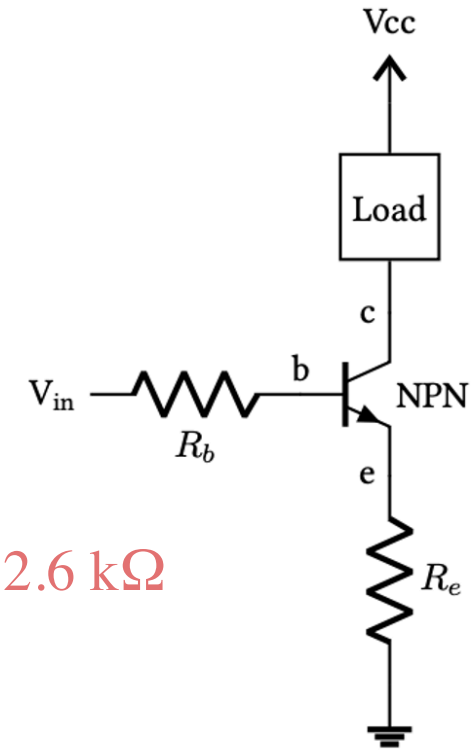
How do we choose R_b and R_e?

Set R_e to zero.

Size R_b from the *load* current and the current gain. For $i_c = 100$ mA and $\beta \approx 100$:

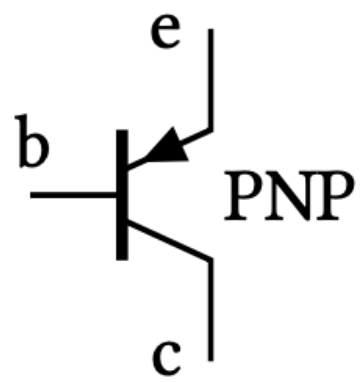
$$i_b = \frac{i_c}{\beta} = \frac{100\text{ mA}}{100} = 1\text{ mA}$$

$$i_b = \frac{V_{in} - 0.7}{R_b} \quad \Longrightarrow \quad R_b = \frac{3.3 - 0.7}{1\text{ mA}} = 2.6\text{ k}\Omega$$

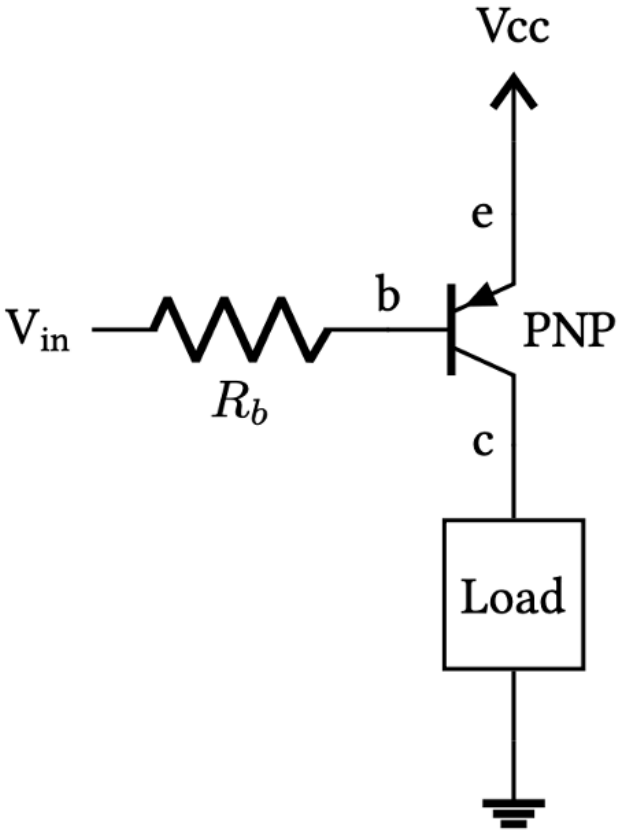


In practice pick something smaller (say $1\text{ k}\Omega$) to overdrive the base and keep the transistor saturated. Note the payoff: **1 mA out of the pin switches 100 mA through the load** – the iCE40 can only source 8 mA.

Driving a load with a PNP transistor



PNP Transistor Symbol



Used to pull load high. (i.e., connect load to power)

Wrap up

- Synchronous sequential design enables us to design simple and robust digital systems.
  - Only one clock signal to all flops (single clock domain)
  - Ensure that the setup and hold time constraints are observed.
- We need to synchronize asynchronous inputs to avoid metastability. Price is an additional clock cycle of latency.
- Transistors are like electrically controlled switches and enable us to drive larger loads from weak source (e.g., FPGA/MCU I/O pins)

Announcements/Reminders

- Checkoffs continue today – don’t delay starting on Lab 2. Can reuse code from Lab 1
  - Only **one** seven_seg Verilog module.
  - Make sure LEDs are consistent brightness no matter how many segments are on
  - Develop a testbench to confirm your circuit is working. See tutorial on the website.